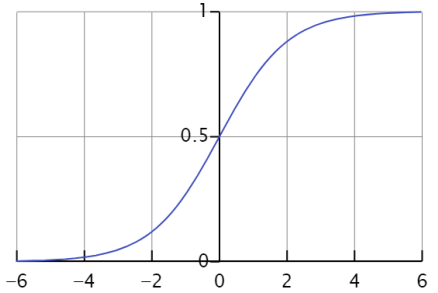
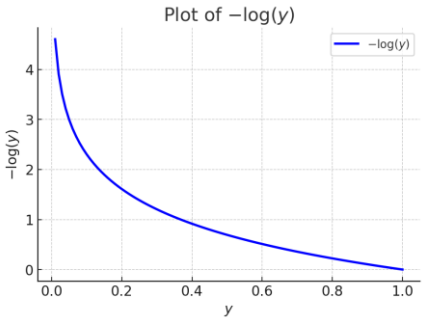
Lecture 4

Logistic regression (Binary classifier):

Logistic regression (Binary classifier):

- Input feature vector $x_i = (x_{i1}, x_{i2}, \dots, x_{id})^T \in \mathbb{R}^d$
- Model parameters (weights) $W = (w_1, \dots, w_d)^T \in \mathbb{R}^d$
- The real valued target value $y_i \in \{0,1\}$
- Bias term $b \in \mathbb{R}$
- Hypothesis function:
  - $\hat{y}_i = f(x_i, W, b) = \sigma(W^T x_i + b) = \frac{1}{1+e^{-(W^T x_i + b)}}$, where $\sigma(z) = \frac{1}{1+e^{-(z)}}$
- Loss function: Cross entropy: $L(W, b) = -\frac{1}{N}\sum_{i=1}^{N}[y_i \log \hat{y}_i + (1 - y_i)\log(1 - \hat{y}_i)]$





- Cross-entropy measures how well the predicted probability distribution aligns with the actual labels.
- For i^{th} data point (x_i): true class y_i and predicted probability $\hat{y}_i$
- **Case 1:** When $y_i = 1$ (Positive class)
  - $L_i(W, b) = -[y_i \log \hat{y}_i + (1 - y_i)\log(1 - \hat{y}_i)] = -\log \hat{y}_i$
  - If $\hat{y}_i = 0.9$ (high predicted probability for class 1)$\Rightarrow L_i(W, b) = -\log(0.9) = 0.10$ (low loss)
  - If $\hat{y}_i = 0.1$ (low predicted probability for class 1))$\Rightarrow L_i(W, b) = -\log(0.1) = 2.3$ (high loss)
- **Case 2:** When $y_i = 0$ (Negative class)
  - $L_i(W, b) = -[y_i \log \hat{y}_i + (1 - y_i)\log(1 - \hat{y}_i)] = -\log(1 - \hat{y}_i)$
  - If $\hat{y}_i = 0.9$ (high predicted probability for class 1, which is wrong)$\Rightarrow L_i(W, b) = -\log(1 - 0.9) = 2.3$ (high loss)
  - If $\hat{y}_i = 0.1$ (low predicted probability for class 1, which is correct)$\Rightarrow L_i(W, b) = -\log(1 - 0.1) = 0.1$ (low loss)

Logistic regression: gradient calculation

- Logistic function: $\sigma(z) = \frac{1}{1+e^{-(z)}}$
- $\frac{d}{dz}\sigma(z) = \frac{d}{dz}\left(1+e^{-(z)}\right)^{-1} = \frac{e^{-z}}{\left(1+e^{-(z)}\right)^2}$

$$= \frac{1}{1+e^{-(z)}}\left[\frac{e^{-z}}{1+e^{-(z)}}\right] = \frac{1}{1+e^{-(z)}}\left[1-\frac{1}{1+e^{-(z)}}\right] = \sigma(z)(1-\sigma(z))$$

- $\hat{y}_i = f(x_i, W, b) = \sigma(W^T x_i + b) = \frac{1}{1+e^{-(W^T x_i + b)}}$
- $\frac{\partial \hat{y}_i}{\partial W} = \hat{y}_i(1-\hat{y}_i)x_i$ % Since $\frac{d}{dz}\sigma(z) = \sigma(z)(1-\sigma(z))$
- $\frac{\partial \hat{y}_i}{\partial b} = \hat{y}_i(1-\hat{y}_i)$

- Now for x_i (single data point, $N = 1$),
  - $L(W,b) = -[y_i \log \hat{y}_i + (1-y_i)\log(1-\hat{y}_i)]$
    - $\frac{\partial L(W,b)}{\partial W} = -\left[y_i \frac{1}{\hat{y}_i}\frac{\partial \hat{y}_i}{\partial W} + (1-y_i)\frac{1}{(1-\hat{y}_i)}\frac{-\partial \hat{y}_i}{\partial W}\right]$
    - $\frac{\partial L(W,b)}{\partial W} = -\left[y_i \frac{1}{\hat{y}_i}\{\hat{y}_i(1-\hat{y}_i)x_i\} - (1-y_i)\frac{1}{(1-\hat{y}_i)}\{\hat{y}_i(1-\hat{y}_i)x_i\}\right]$
    - $\frac{\partial L(W,b)}{\partial W} = [-y_i\{(1-\hat{y}_i)x_i\} + (1-y_i)\{\hat{y}_i x_i\}]$
    - $\frac{\partial L(W,b)}{\partial W} = (\hat{y}_i - y_i)x_i$
  - $L(W,b) = -[y_i \log \hat{y}_i + (1-y_i)\log(1-\hat{y}_i)]$
    - $\frac{\partial L(W,b)}{\partial b} = -\left[y_i \frac{1}{\hat{y}_i}\frac{\partial \hat{y}_i}{\partial b} + (1-y_i)\frac{1}{1-\hat{y}_i}\left(-\frac{\partial \hat{y}_i}{\partial b}\right)\right]$
    - $\frac{\partial L}{\partial b} = -\left[y_i \frac{1}{\hat{y}_i}(\hat{y}_i(1-\hat{y}_i)) - (1-y_i)\frac{1}{1-\hat{y}_i}(\hat{y}_i(1-\hat{y}_i))\right] = (\hat{y}_i - y_i)$

- For N data points:
  - $L(W,b) = -\frac{1}{N}\sum_{i=1}^{N}[y_i \log \hat{y}_i + (1-y_i)\log(1-\hat{y}_i)]$
  - $\frac{\partial L(W,b)}{\partial W} = \frac{1}{N}\sum_{i=1}^{N}(\hat{y}_i - y_i)x_i = \frac{1}{N}X^T(\hat{Y}-Y) = \frac{1}{N}X^T E$
  - Similarly:
  - $\frac{\partial L(W,b)}{\partial b} = \frac{1}{N}\sum_{i=1}^{N}(\hat{y}_i - y_i) = \frac{1}{N}1^T(\hat{Y}-Y) = \frac{1}{N}1^T E$

Assignment question:

- Given the following data for **logistic regression** problem:
- Feature matrix $X = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$, Weight vector $W = \begin{bmatrix} 0.5 \\ 0.5 \end{bmatrix}$, Bias term $\mathrm{b} = 1$, Target vector $Y = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$,
- Learning rate $\alpha = 0.01$,
- Logistic regression details:
  - Hypothesis function $\hat{y}_i = f(x_i, W, b) = \sigma(W^T x_i + b) = \frac{1}{1+e^{-(W^T x_i + b)}}$
  - Loss function: Cross entropy: $L(W, b) = -\frac{1}{N}\sum_{i=1}^{N}[y_i \log \hat{y}_i + (1 - y_i)\log(1 - \hat{y}_i)]$
- Perform 2 iterations of GD and SGD to update the weights W and bias b. Show all intermediate steps, including predictions, errors, gradient calculations, Loss function values and weight updates for each iteration.

Adaptive step size based GD algorithms:

- Adagrad
- RMSProp
- Adam

Adaptive step size based GD algorithms:

$$X = \begin{bmatrix} 1.2589 & 1.5946 \\ 1.6232 & 0 \\ -1.4921 & 0 \\ 1.6535 & 0 \\ 0.5294 & 0.7739 \\ -1.6098 & 0 \\ -0.8860 & -0.5226 \\ 0.1875 & 0.0358 \\ 1.8300 & 0 \\ 1.8596 & 0 \end{bmatrix}$$

Sparse dataset

- **Sparse Data Problem**: In machine learning problems with sparse data (where many features might have zero or near-zero values for many data points), traditional gradient descent struggles to handle rare features because all features share the same learning rate.

- **Fixed Learning Rate Issue:**
- In traditional GD, selecting an optimal fixed learning rate is a challenge. A fixed learning rate may work well for certain dimensions of the parameter space but poorly for others.
- Using a fixed learning rate throughout the training process can lead to suboptimal performance, as it may be too large or too small at different stages of learning
- The choice of the initial learning rate can significantly affect convergence speed and stability.

Requirement of Multi-dimensional step size in d- dimensional feature space.

Adagrad:

Dimensional wise or feature wise learning rate

Intuition:. The core idea is to perform larger updates for parameters associated with infrequent features and smaller updates for parameters associated with frequent features.

Memory based mechanism

Adagrad

%Inputs

$L(W)$: $\mathbb{R}^n \rightarrow \mathbb{R}$, a smooth loss function

$W^{(0)} \in \mathbb{R}^n$ The initial weight parameters

$G^{(0)} = 0_n \in \mathbb{R}^n$: Archive for Accumulated square gradient

$\alpha \in \mathbb{R}$: Learning rate

ε: a small positive value

%Output: $W^* \in \mathbb{R}^n$ optimal weight vector

Begin

$k \leftarrow 1$

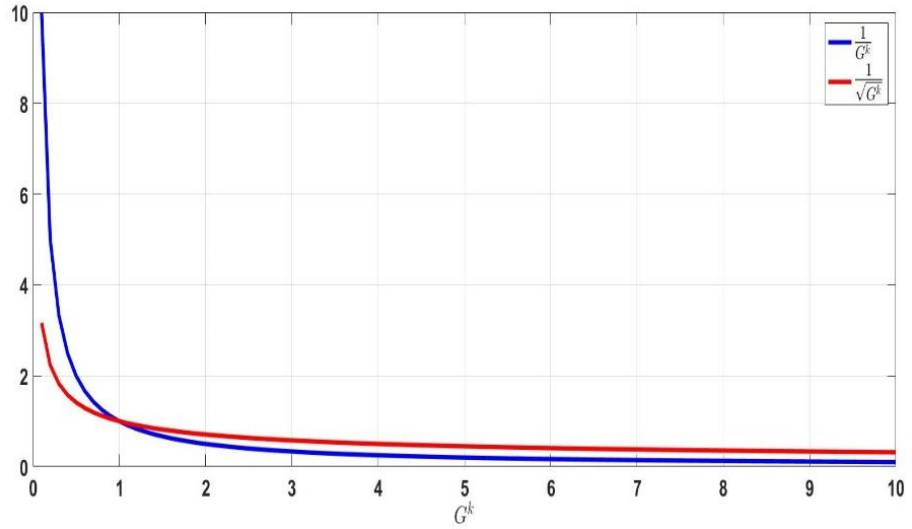
While (stopping criteria $\left\|\nabla_W L\left(W^{(k)}\right)\right\| > \varepsilon$) **do**

- Compute gradient $\nabla_W L\left(W^{(k)}\right)$
- $G^{(k)} = G^{(k-1)} + \left(\nabla_W L\left(W^{(k)}\right)\right)^2$ % Accumulate square gradient
- $W^{(k+1)} = W^{(k)} - \frac{\alpha}{\sqrt{G^{(k)}+\varepsilon}} \nabla_W L\left(W^{(k)}\right)$ %Update position
- $k = k + 1;$

EndWhile

End

Why square root?
For controlled exploration



Adagrad:

Adagrad for 2 D feature space

%Inputs

$L(W): \mathbb{R}^2 \to \mathbb{R}$, a smooth loss function

$W^{(0)} = (w_1^{(0)}, w_2^{(0)}) \in \mathbb{R}^2$ The initial weight parameters

$G^{(0)} = \left(g_1^{(0)}, g_2^{(0)}\right) = (0,0) \in \mathbb{R}^2$: Accumulated square gradient

$\alpha \in \mathbb{R}$: Learning rate

ε: a small positive value

%Output: $W^* \in \mathbb{R}^2$ optimal weight vector

Begin

$k \leftarrow 1$

While (stopping criteria $\left\|\nabla L\left(w_1^{(k)}, w_2^{(k)}\right)\right\| > \varepsilon$) **do**

- Compute gradient $\nabla L\left(w_1^{(k)}, w_2^{(k)}\right) = \left[\frac{\partial L}{\partial w_1^{(k)}}, \frac{\partial L}{\partial w_2^{(k)}}\right]^T$
- $\left(g_1^{(k)}, g_2^{(k)}\right) = \left(g_1^{(k-1)}, g_2^{(k-1)}\right) + \left(\left[\frac{\partial L}{\partial w_1^{(k)}}, \frac{\partial L}{\partial w_2^{(k)}}\right]^T\right)^2$ % Accumulate square gradient
- $\left(w_1^{(k+1)}, w_2^{(k+1)}\right) = (w_1^{(k)}, w_2^{(k)}) - \alpha \frac{1}{\left(\sqrt{g_1^{(k)}+\varepsilon}, \sqrt{g_2^{(k)}+\varepsilon}\right)} \left[\frac{\partial L}{\partial w_1^{(k)}}, \frac{\partial L}{\partial w_2^{(k)}}\right]^T$ %Update position
- $k = k + 1;$

EndWhile

End

- If first feature is frequently updated:

$$w_1^{(k+1)} = w_1^{(k)} - \left(\frac{1}{\sqrt{g_1^{(k)} + \varepsilon}}\right) \alpha \times \frac{\partial L}{\partial w_1^{(k)}};$$

% Small step size to **frequently updated feature**

- If second feature is sparse nature:

$$w_2^{(k+1)} = w_2^{(k)} - \left(\frac{1}{\sqrt{g_2^{(k)} + \varepsilon}}\right) \alpha \times \frac{\partial L}{\partial w_2^{(k)}};$$

% Large step size to **not frequently updated feature**

Adagrad:

- **Limitations:**
- **Aggressive decay in step size**: The component of $G^{(k)}$ is strictly non decreasing. The accumulated sum causes the effective learning rate to decrease during training, often becoming infinitesimally small before convergence.
- Frequently updated parameters will get very small step sizes in the terminal phase of the search
- **Accumulation of Squared Gradients**: Storage and stagnation issues
- No decay coefficient for controlling the accumulation rate as per the search requirement.
- Not well-suited for non-convex problems:

Adagrad: assignment question

- Given the following data for **logistic regression** problem:
- Feature matrix $X = \begin{bmatrix} 1 & 0.2 \\ 3 & 0.01 \end{bmatrix}$, Weight vector $W = \begin{bmatrix} 0.5 \\ 0.5 \end{bmatrix}$, Bias term b $= 1$, Target vector $Y = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$,
- Learning rate $\alpha = 0.01$,
- $\varepsilon = 10^{-8}$ (for numerical stability)
- Logistic regression details:
  - Hypothesis function $\hat{y}_i = f(x_i, W, b) = \sigma(W^T x_i + b) = \frac{1}{1+e^{-(W^T x_i + b)}}$
  - Loss function: Cross entropy: $L(W, b) = -\frac{1}{N}\sum_{i=1}^{N}[y_i \log \hat{y}_i + (1 - y_i)\log(1 - \hat{y}_i)]$
- Perform 2 iterations of **Adagrad** to update the weight vector W and bias b. Show all intermediate steps, including predictions, errors, gradient calculations, accumulator updates, and parameter updates for each iteration.